

Chương 2. Danh sách - LIST

Tóm tắt :

1. Khái niệm
2. Cài đặt danh sách tuyến tính bằng mảng
3. Danh sách liên kết đơn

2.1. Khái niệm danh sách - LIST

Danh sách là một dãy các phần tử, mỗi phần tử gọi là một nút (node).

Khi các nút chỉ lưu trữ các thông tin dữ liệu và được lưu trữ một cách tuần tự thì gọi là danh sách thường hay tuyến tính, kiểu danh sách này được xử lý theo tuần tự, tuyến tính.

Khi các nút lưu trữ các thông tin dữ liệu và địa chỉ của nút kế tiếp thì gọi là danh sách liên kết đơn.

Khi các nút lưu trữ thông tin dữ liệu và 2 địa chỉ liên kết với nút trước và nút sau nó thì gọi là danh sách liên kết đôi.

Danh sách được tổ chức, cấu trúc theo kiểu dữ liệu có tên kiểu là **list**.

Kiểu dữ liệu **list** dùng cho danh sách là một trong các kiểu dữ liệu mà được hỗ trợ bởi thư viện Standard Template Library (STL) trong C++, khi trong chương trình có dùng biến kiểu list thì phải viết dòng `#include <list>` vào phần đầu chương trình.

Cấu trúc dữ liệu **List** được hỗ trợ bởi STL là một danh sách liên kết đôi, khác với kiểu cấu trúc **Vector** là kiểu mảng động (dynamic array), là mảng được cấp phát bộ nhớ động bằng **Allocator** (đã học ở phần Con trỏ).

Trong thực tế, một danh sách thông thường sẽ dùng cấu trúc dữ liệu Vector, các thao tác xử lý chủ yếu là thêm phần tử vào cuối mảng và truy cập phần tử theo theo ngẫu nhiên (địa chỉ) nên nhanh và dễ. Tuy nhiên, việc thêm phần tử vào cuối mảng sẽ nhanh hơn việc chèn thêm phần tử vào đầu hay giữa mảng do việc trước khi chèn vào vị trí k, phải lùi các phần tử từ vị trí k đến cuối mảng ra sau 1 vị trí.

Danh sách liên kết đôi chỉ nên dùng khi phải thực hiện nhiều lần các phép chèn/xóa phần tử vào đầu hay giữa danh sách, ít phải truy nhập ngẫu nhiên tới các phần tử. Việc chèn/xóa phần tử trong danh sách liên kết đôi chỉ đơn giản là thay đổi thông tin liên kết ở 2 bên trái-phải của nút. Tuy nhiên, vì các nút không có địa chỉ index thực nên việc truy nhập các phần tử nút chậm, phải duyệt danh sách cho đến khi gặp được phần tử cần thiết.

Những thao tác cơ bản đối với danh sách :

1. Khai báo cấu trúc nút, cấu trúc danh sách
2. Tạo một danh sách rỗng ban đầu, tạo một **nút mới** nhận dữ liệu
3. Thêm **nút mới** vào danh sách
4. Xóa một phần tử từ danh sách
5. Duyệt, xử lý các phần tử danh sách

2.2.Cài đặt danh sách tuyến tính bằng mảng

Cách sử dụng thông thường nhất đối với danh sách là coi nó như một mảng tĩnh –array hay một mảng động – vector, có dùng đến con trỏ.

Trong mục này xét danh sách như một mảng tĩnh, dùng mảng để cài đặt danh sách (đã học trong môn Lập trình bằng C++)

2.2.1.Khai báo danh sách

Cú pháp :

```
#define Max <số phần tử lớn nhất của dãy >;  
        < kiểu dữ liệu> <tên danh sách>[Max];
```

Sau khi khai báo số phần tử tối đa của danh sách Max, tiếp đến cần khai báo kiểu dữ liệu của các phần tử trong danh sách, kiểu dữ liệu vị trí từng phần tử, nếu mỗi phần tử có kiểu cấu trúc struct thì phải định nghĩa struct.

a)Cú pháp khai báo kiểu cấu trúc

```
struct <tên kiểu cấu trúc> // <tên kiểu cấu trúc> do ta tự đặt  
{ <kiểu dữ liệu> <tên trường dữ liệu thành phần>;  
  .....  
  <kiểu dữ liệu> <tên trường dữ liệu thành phần >;  
};
```

Ví dụ . với bài toán quản lý sv , có thể khai báo kiểu cấu trúc sinh viên như sau :

```
struct sinhvien  
{   char masv[10];  
    string hoten;  
    int ngaysinh;//  
    float diem[15];//điểm của 15 môn của 1 sv  
    float diemtb;  
};
```

Phiếu tin
của 1 sv

b)Khai báo biến cấu trúc

Sau khi khai báo **kiểu cấu trúc**, để sử dụng được kiểu cấu trúc đó phải khai báo **biến cấu trúc**, là biến có kiểu cấu trúc đã được định nghĩa trước đó.

Có 2 cách khai báo biến cấu trúc.

Cách 1: tên biến cấu trúc được viết ngay sau dấu } kết thúc định nghĩa kiểu cấu trúc .

Ví dụ. đặt tên biến sv có kiểu cấu trúc sinhvien :

```
struct sinhvien  
{ char masv[10];  
  string hoten;  
  int ngaysinh;  
  float diem[10];  
  float diemtb;  
}sv;
```

Cách 2. tên biến kiểu cấu trúc được viết ở dòng riêng biệt theo cú pháp :

struct <kiểu-cấu-trúc> < tên-biến-cấu trúc>;

Ví dụ.

```
struct sinhvien
{
    char masv[10];
    string hoten;
    int ngaysinh;
    float diem[10];
    float diemtb;
};
struct sinhvien sv;      //sv là biến cấu trúc có kiểu cấu trúc sinhvien
```

Chương trình chỉ làm việc với **tên biến cấu trúc**, tên kiểu cấu trúc chỉ để xác định **kiểu dữ liệu cấu trúc** của biến.

c) Cài đặt danh sách tuyến tính bằng mảng cấu trúc 1 chiều

1) Khai báo mảng kiểu cấu trúc

Sau khi có khai báo kiểu và biến cấu trúc, muốn sử dụng mảng cấu trúc thì có thể khai báo theo 1 trong 2 cách giống như khai báo biến cấu trúc :

Cách 1.

```
struct sinhvien
{
    char masv[10];
    string hoten;
    int ngaysinh;
    float diem[10];
    float diemtb;
}sv,dslop[60];
```

// dslop[60] là mảng có 60 phần tử, mỗi phần tử có kiểu cấu trúc **sinhvien**

Cách 2. Khai báo riêng 1 dòng :

```
_struct sinhvien
{
    char masv[10];
    string hoten;
    int ngaysinh;
    float diem[10];
    float diemtb;
};
struct sinhvien sv,dslop[60];
```

Trong dòng khai báo này, **sv** là biến cấu trúc **sinhvien**, **dslop[60]** là tên mảng có kiểu cấu trúc **sinhvien** có tối đa 60 phần tử , mỗi phần tử có kiểu cấu trúc **sinhvien**

2) Truy cập phần tử mảng kiểu cấu trúc

Để truy cập các thành phần của phần tử thứ i của một mảng kiểu cấu trúc, ta viết :

Tên_mảng[i].Tên-Thành-Phần;

Ví dụ : `dslop[i].masv, dslop[i].hoten, // mã và họ tên sv thứ  $i$  trong dslop`

Ví dụ : `struct sinhvien vl64[60];`

Ví dụ áp dụng danh sách là mảng tĩnh

Xét bài toán cộng 2 đa thức th- a :

$$A = \sum a_i x^i = 5x^{970} + 12x^{400} + 7$$

$$B = \sum b_j x^j = x^{15} + 8x^3 + 1$$

Lập chương trình tính $C=A+B$

Ta tổ chức dữ liệu nh- sau :

Mảng 1(term1):

Hệ số →	5	12	7
Số mũ →	970	400	0

Mảng 2(term2) :

Hệ số →	1	8	1
Số mũ →	15	3	0

Mảng 3(term3) :

Hệ số →							
Số mũ →							

Thuật toán tính hệ số của đa thức kết quả $C=A+B$:

$$c[k] = \begin{cases} a[i] & \text{nếu số mũ của } a[i] > \text{số mũ của } b[j] \\ b[j] & \text{nếu số mũ của } a[i] < \text{số mũ của } b[j] \\ a[i]+b[j] & \text{nếu số mũ của } a[i] = \text{số mũ của } b[j] \end{cases}$$

Khai báo cấu trúc các nút của danh sách là các phần tử của một đa thức :

```
struct node
{
    int heso; // nút có 2 thành phần dữ liệu
    int somu; // là hệ số, số mũ
};
```

Khai báo danh sách gồm các nút có cấu trúc node :

```
struct node term1[20], term2[20], term3[40];
```

Ở đây, `term1[20], term2[20], term3[40]` là 3 danh sách tuyến tính, mỗi phần tử của các danh sách có kiểu `struct node`.

Bài tập về nhà : viết chương trình C++ cho bài toán trên

2.3. Danh sách liên kết đơn

2.3.1. Giới thiệu

Chúng ta đã xét loại danh sách tuyến tính, được lưu trữ tuần tự. Đa thức đ-ợc sắp xếp theo trật tự số mũ, các ma trận đ-ợc bố trí theo trật tự hàng cột. Chúng ta đã sử dụng mảng để cài đặt các danh sách tuyến tính, trong ph-ơng pháp này ngầm hiểu thứ tự của mảng cài đặt là thứ tự xử lý của các phần tử dữ liệu. Do cách cài đặt này mà mỗi khi chèn hay xoá một phần tử chúng ta phải dịch chuyển các phần tử khác để xác định lại trật tự của mảng. Điều này rất tốn thời gian trong tr-ờng hợp phải th-ờng xuyên chèn /xoá các phần tử.

Để khắc phục điều này, chúng ta sử dụng cấu trúc dữ liệu kiểu danh sách liên kết đơn (*trong mục 2.3 của chương này khi nói danh sách liên kết là nói về danh sách liên kết đơn*). Nếu nh- trong danh sách tuyến tính, thứ tự các phần tử đ-ợc ngầm hiểu, đ-ợc ẩn, thì ng-ợc lại trong danh sách liên kết, thứ tự các phần tử ở dạng hiển, đ-ợc biểu diễn rõ.

Một danh sách liên kết là một dãy, một tập các nút , mỗi nút có 2 phần là phần *Dữ liệu* và phần *Liên kết* :

Dữ liệu	Liên kết
---------	----------

Ví dụ. Giả thiết ta có danh sách l-u trong mảng 1 chiều :

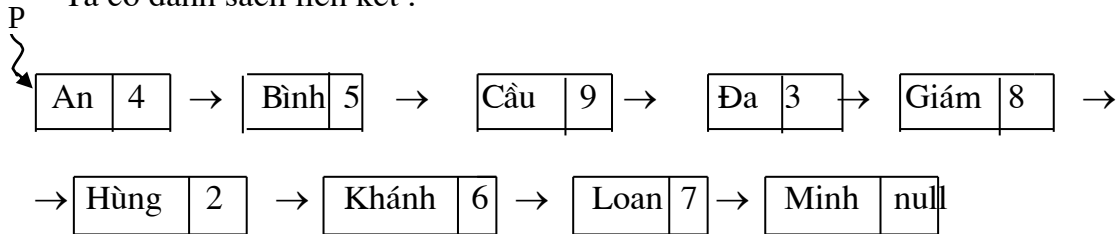
1	An
2	Khánh
3	Giám
4	Bình
5	Cầu
6	Loan
7	Minh
8	Hùng
9	Đa

Để thiết lập trật tự họ tên theo vần ABC mà không cần phải dịch chuyển lại các phần tử, ta thêm 1 cột chỉ mỗi liên kết:

	<i>Du_Lieu</i>	<i>Lien_ket</i>
1	An	4
2	Khánh	6
3	Giám	8
4	Bình	5
5	Cầu	9
6	Loan	7
7	Minh	Null
8	Hùng	2
9	Đa	3

Giá trị các phần tử trong cột *Liên kết* này sẽ trỏ đến các phần tử trong danh sách. Trong danh sách, bắt đầu từ phần tử có $Du_Lieu[1] = AN$, ta đặt một biến con trỏ P có giá trị trỏ đến phần tử ban đầu. Theo trật tự, sau An là Bình ở $Du_Lieu[4]$, nên để trỏ đến Bình thì $LienKet[1] = 4$; sau Bình là Cầu ở $Du_Lieu[5]$ nên $LienKet[4] = 5$, ..., cứ thế cho đến sau Loan ở $Du_Lieu[6]$ là Minh ở $Du_Lieu[7]$ nên $LienKet[6] = 7$, sau Minh ở $Du_Lieu[7]$ không có gì nên ta đặt $LienKet[7] = Null$.

Ta có danh sách liên kết :



Với kiểu danh sách này khi chèn hay xóa một phần tử không cần phải dịch chuyển các phần tử khác trong danh sách.

Để cấu trúc danh sách liên kết, tr- ớc hết phải định vị đ- ợc phần tử đầu tiên trong danh sách. Khi cho vị trí của một nút bất kỳ trong danh sách, ta có thể xác định đ- ợc nút mà nó liên kết tới.

Một danh sách liên kết gồm:

- Một tập các nút

- Mỗi nút là một bản ghi trong đó có hai phần : phần các tr- ờng dữ liệu và phần tr- ờng liên kết-con trỏ móc nối.

Phần dữ liệu của mỗi nút l- u trữ dữ liệu, phần con trỏ liên kết sẽ chỉ đến nút tiếp theo. Khi ở cuối danh sách, con trỏ liên kết không chỉ vào đâu cả, ta nói rằng nó là rỗng (null)

Nếu $*p$ là con trỏ, trỏ đến địa chỉ chứa một nút nào đó trong danh sách liên kết, ta ký hiệu phần dữ liệu của nút này là $Du_Lieu(p)$, phần liên kết là $LienKet(p)$. Để tạo một danh sách rỗng ban đầu ta gán giá trị null vào đầu danh sách. Để kiểm tra một danh sách rỗng hay không ta chỉ việc xác định đầu danh sách có giá trị Null hay không.

2.3.2.Cài đặt danh sách liên kết

Danh sách liên kết đơn là một cấu trúc dữ liệu động, được tạo nên nhờ việc cấp phát động nên nó có một số đặc điểm sau đây:

- Được cấp phát bộ nhớ khi chạy chương trình
- Có thể thay đổi kích thước qua việc thêm, xóa phần tử
- Kích thước tối đa phụ thuộc vào bộ nhớ khả dụng của RAM
- Các phần tử được lưu trữ ngẫu nhiên (không liên tiếp) trong RAM
- Chỉ cần nắm được phần tử đầu và cuối là có thể quản lý được danh sách liên kết
- Truy cập tới phần tử ngẫu nhiên phải duyệt từ đầu đến vị trí đó
- Chỉ có thể tìm kiếm tuyến tính một phần tử

2.3.2.1.Khai báo cấu trúc nút và cấu trúc danh sách liên kết

Danh sách liên kết đơn được tạo thành từ các nút(node),. mỗi nút gồm hai thành phần là dữ liệu và liên kết. Thành phần dữ liệu có thể là kiểu dữ liệu cơ bản hoặc tự định nghĩa, phần liên kết là con trỏ, trỏ đến nút tiếp theo.

a) Khai báo kiểu nút :

```
struct Node
{
    int du_lieu;
    Node* next; // khai báo kiểu đệ quy
};
```

Chú ý : đây là khai báo tên kiểu cấu trúc *Node*, trong đó có 2 thành phần : *du_lieu* và liên kết *next*, liên kết *next* được khai báo theo đệ quy, nghĩa là *next* là con trỏ kiểu *Node* cũng có 2 thành phần *du_lieu* và con trỏ *next* tiếp...

b) Khởi tạo nút

Để khởi tạo một nút mới, cần cấp phát bộ nhớ động cho nó, khởi tạo giá trị ban đầu và trả về nút mới được cấp phát.

```
Node* KhoitaoNode(int du_lieu0) // du_lieu0 là khởi tạo giá trị ban đầu cho nút mới
{
    Node* node = new Node; // tạo node mới và cấp phát bộ nhớ động cho node
    node->du_lieu = du_lieu0; // khởi gán giá trị dữ liệu ban đầu du_lieu
    node->next = NULL; // nút mới tạo chưa trỏ vào đâu cả nên next=NULL
    return node; // trả về nút mới tạo
}
```

2.3.2.2. Khai báo và tạo danh sách liên kết đơn

Sau khi tạo cấu trúc nút của danh sách liên kết đơn, việc tiếp theo là tạo cấu trúc **List** lưu trữ phần tử đầu (head) và phần tử cuối (hay đuôi tail).

a) Khai báo kiểu cấu trúc danh sách liên kết

```
struct List
{
    Node* head;
    Node* tail;
};
```

b) Hàm khởi tạo danh sách liên kết rỗng ban đầu

```
void KhoiTaoList(List& list)
{
    list.head=NULL; // Lúc đầu danh sách rỗng head và tail chưa trỏ vào đâu cả
    list.tail=NULL; //, được gán NULL
}
```

Trong phần hàm main(), đặt một con trỏ **list** có kiểu cấu trúc struct List và tạo một danh sách liên kết đơn bằng hàm void KhoiTaoList(List& list) :

```
List list;
KhoiTaoList(list);
```

kết quả đã khởi tạo được danh sách liên kết với con trỏ list có kiểu cấu trúc struct List, head và tail được gán NULL.

2.3.2.3. Thêm nút vào danh sách liên kết

Trước khi thêm nút vào danh sách, phải khởi tạo một nút bằng hàm `KhoitaoNode(int du_lieu0)`, hàm này trả về một node đã có phần dữ liệu, còn liên kết chưa trở vào đâu cả.

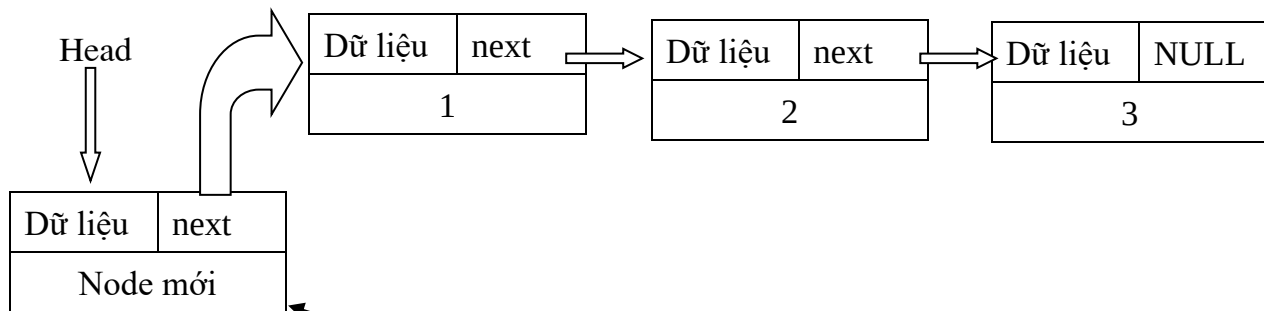
a) Thêm nút vào đầu danh sách

Để thêm node vào đầu danh sách, cần làm 2 bước :

✓ Kiểm tra nếu danh sách rỗng (`head == NULL`) thì thêm nút vào đầu danh sách, gán `head = node` ; `tail = node`.

✓ Nếu danh sách không rỗng thì cho `node->next` trở vào `head` và node thay thế `list.head` của danh sách.

Ta có sơ đồ :



Hàm :

```
void ThemvaoDau(List& list, Node* node)
{ if(list.head == NULL)
    { list.head = node;
      list.tail = node;
    }
  else
    { node->next = list.head;
      list.head = node;
    }
}
```

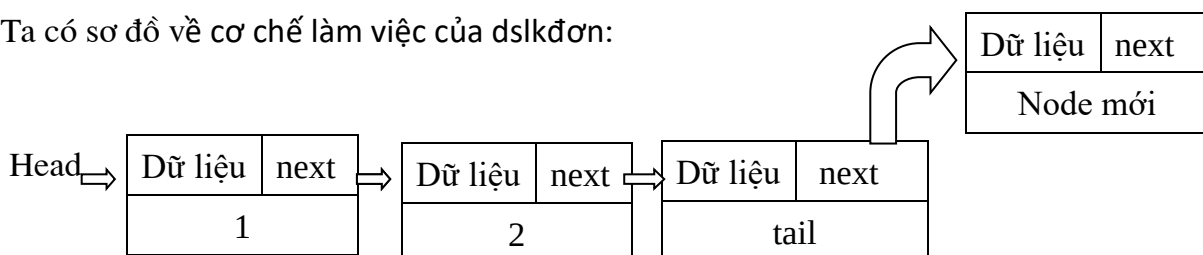
b) Thêm nút vào cuối danh sách

Để thêm node được khởi tạo bởi `KhoitaoNode(int du_lieu0)` vào cuối danh sách liên kết, thực hiện :

✓ Kiểm tra nếu danh sách rỗng (`head == NULL`) thì thêm nút vào cuối danh sách cũng tương tự như thêm vào đầu danh sách, gán `head = node` ; `tail = node`.

✓ Nếu danh sách không rỗng, cho con trỏ `tail->next` trở vào node mới, sau đó gán lại `tail` của danh sách bằng node mới.

Ta có sơ đồ về cơ chế làm việc của `dsliđơn`:



Hàm :

```
void ThemvaoCuoi(List& list, Node* node)
{ if(list.head == NULL)
    { list.head = node;
      list.tail = node;
    }
  else
    { list.tail->next = node ; //next của tail trở vào node
      list.tail = node; //node đóng vai trò tail
    }
}
```

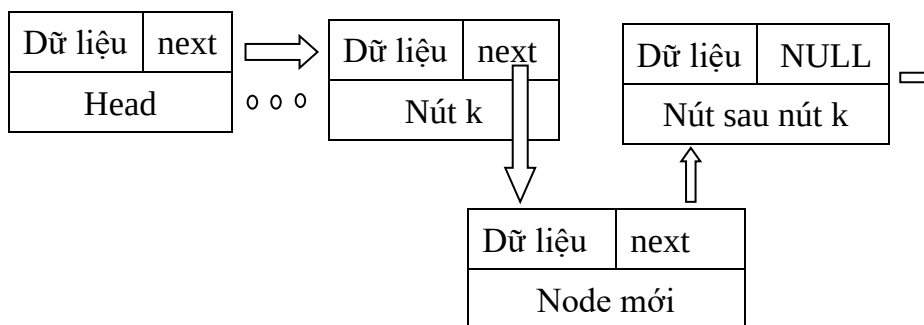
c) Thêm node vào sau nút k (k là tên nút chứ không phải là số thứ tự)

c1. Để thêm một node mới vào giữa danh sách, sau một nút k bất kỳ, đầu tiên ta cần kiểm tra xem danh sách có rỗng không, nếu rỗng thì sẽ thêm vào đầu danh sách.

c2. Kiểm tra tiếp xem nút k có phải là cuối hay không, nếu nút k là cuối thì thêm node vào cuối như ở mục **b)**

c3. Nếu cả c1 và c2 đều không đúng, nút k tồn tại, ta thêm node vào ở giữa danh sách, thực hiện $node \rightarrow next = k \rightarrow next$, sau đó $k \rightarrow next = node$ mới

. Ta có sơ đồ :



Hàm :

```
void ThemNutSauK(List& list, Node* node Node* k)
{ if(k != NULL)
    { node->next = k->next;
      k->next = node->next;
      if(list.tail == k) //nếu k là nút đuôi - tail
        list.tail = node; // thì node mới sẽ là đuôi - tail
    }
  else
    ThemvaoDau(list, node) ; // nếu k không ở giữa, không ở đuôi thì thêm vào đầu
}
```

d) Xóa nút

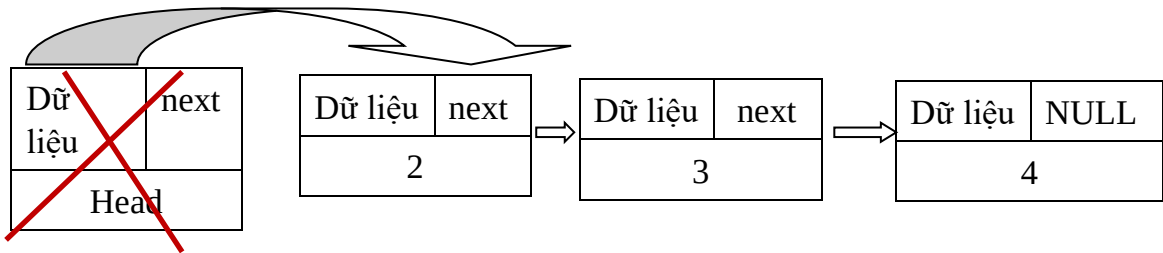
d1. Xóa nút đầu

Bước 1. Kiểm tra xem danh sách đó có rỗng hay không, nếu rỗng, trả về kết quả là 0 và thoát ra nếu không rỗng thì tiếp bước 2;

Bước 2. Lưu lại nút đầu, gán head bằng next của nút đầu rồi mới xóa nút đầu;

Bước 3. Kiểm tra danh sách vừa bị xóa nút đầu có rỗng không, nếu rỗng gán tail = NULL rồi trả về kết quả 1.

. Ta có sơ đồ cơ chế làm việc của dslk đơn :



Hàm :

```
int XoaNutDau(List& list, int& a)
{
    if(list.head != NULL)
    {
        Node* node = list.head; //lưu lại nút đầu
        a = node->du_lieu; //lấy dữ liệu của nút sẽ bị xóa
        list.head = node->next; //đầu của danh sách sau khi xóa sẽ là nút kế tiếp nút đầu
        delete node; //xóa nút lưu nút đầu
        if(list.head == NULL) list.tail = NULL;
        return 1;
    }
    return 0;
}
```

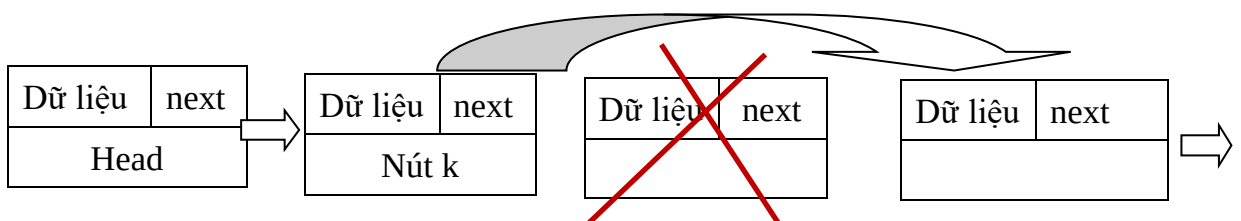
(Nếu vẽ sơ đồ khối thuật toán thì phải dùng các ký pháp sơ đồ khối trong Tin học đại cương)

d1. Xóa nút sau một nút k xác định (k là tên nút chứ không phải chỉ số)

Để xóa một nút sau một nút xác định k, cần làm :

- ✓ Kiểm tra nút k có NULL hay không, nếu k == NULL thì không xóa, trả về 0,. Nếu nút k khác NULL nhưng next của k là NULL thì cũng không xóa, trả về 0.
- ✓ Nếu có nút sau k, kiểm tra tiếp xem nút sau k có phải là tail hay không, nếu ĐÚNG thì gán lại tail là k, và xóa nút sau k.

. Ta có sơ đồ :



Trong sơ đồ trên, ta thực hiện xóa nút k+1 đứng sau nút k. Sau khi xóa, next của nút k sẽ trở vào next của nút bị xóa, tức là trở sang nút thứ k+2, sau đó xóa nút k+1.

```
int XoaNutSauK(List& list, Node* k, int& a)
{
    if(k != NULL)
    {
        Node* node = k->next; // node là nút sau k
        if (node != NULL)
        {
            if (list.tail == node) list.tail =k;
            k->next = node->next;
            a = node->du_lieu;
            delete node;
            return 1;
        }
        return 0;
    }
    return 0;
}
```

e) Hiện danh sách

Hiện danh sách là truy nhập đến từng nút và xử lý phần Du_Lieu. Có nhiều phép xử lý khác nhau, để đơn giản ta chọn việc hiện dữ liệu lên màn hình.

Để hiện danh sách lên màn hình, ta duyệt từ đầu đến cuối danh sách và hiện ra dữ liệu của nút. Bắt đầu từ nút head, kiểm tra nút đó có NULL hay không để hiện dữ liệu nút, sau đó gán tiếp node bằng next của chính nó tức là đến nút tiếp theo cho đến hết danh sách.

```
void HienDS(List list)
{
    if (list.head != NULL)
    {
        Node* node = list.head;
        while (node != NULL)
        {
            cout<< node->du_lieu<<" ";
            node = node->next;
        }
    }
}
```

f) Hiện node bất kỳ theo chỉ số nút

Để hiện nút trong danh sách theo chỉ số nút, thực hiện tương tự như khi hiện danh sách. Ta sẽ tạo một biến đếm để biết vị trí hiện thời, duyệt qua các node cho đến khi node bằng NULL hoặc biến đếm trùng khớp với vị trí node cần hiện. Kiểm tra xem nếu node khác NULL và biến đếm bằng vị trí cần hiện, ta sẽ trả về node đó, ngược lại trả về NULL (danh sách rỗng hoặc là vị trí cần hiện nằm ngoài phạm vi của danh sách).

```
Node* HienNut(List& list, int chi_so) // tìm và hiện nút theo chỉ số
```

```

{Node* node = list.head;
int i=0;
while (node != NULL && i != chi_so)
    {node = node->next; i++; }
if(i== chi_so && node != NULL)
    return node;
return NULL;
}

```

i)Tìm kiếm phần tử trong danh sách theo dữ liệu

Tìm kiếm nút theo giá trị dữ liệu cũng là duyệt danh sách từ đầu cho đến khi tìm thấy nút có giá trị trùng khớp hoặc đến hết danh sách (node ==NULL) mà không thấy, nếu tìm thấy node, sẽ trả về node đó.

```

Node* TimNut(List& list, int a) //tìm theo dữ liệu
{ Node* node = list.head;
while (node != NULL && node->du_lieu != a)
    node = node->next;
if(node != NULL)
    return node;
return NULL;
}

```

j)Đếm số phần tử của danh sách

Để đếm số phần tử ta duyệt từ đầu đến cuối danh sách và đếm số node.

```

int Count(List& list)
{ int n=0;
Node* node = list.head;
while (node != NULL )
    {n++;
node = node->next;
}
return n;
}

```

k)Xóa danh sách

Để xóa danh sách, ta duyệt từ đầu danh sách và hủy từng node theo hàm XoanutDau cho đến khi node == NULL. Sau khi xóa hết tất cả phần tử thì gán lại tail bằng NULL.

```

void XoaDS(List& list)
{ int a;
Node* node = list.head;

```

```

while (node != NULL)
    { XoaNutDau(list, a);
      node = list.head; }
list.tail = NULL;
}

```

Bài tập : học kỹ các khai báo nút, danh sách liên kết, các hàm tạo nút, thêm nút, xóa nút, hiện nút, hiện danh sách... viết thêm hàm main() thành một chương trình về danh sách liên kết đơn có các chức năng :

- Khai báo list
- Khởi tạo list
- Khởi tạo node
- Dùng lệnh lặp for và hàm KhoitaoNode(i), ThemvaoCuoi(list, node) để tạo 5 nút đầu danh sách liên kết có giá trị dữ liệu là các số lẻ int a[5] = {1, 3, 5, 7, 9}
- Hiện danh sách : HienDS(list);
- Hiện nút theo vị trí số 3 : nut3= HienNut(list, 3);
- Tìm nút theo giá trị 7 : nut7= TimNut(list, 7);
- Xóa nút sau nút nut7 : XoaNutSauK(list, nut7, a);
- Hiện danh sách sau khi xóa nut7 : HienDS(list);
- Tạo thêm nút có giá trị 8 : Node* nut8 = KhoitaoNode(8);
- Thêm nut8 vào sau nut3 : ThemNutSauK(list, nut8, nut3);
- Hiện danh sách sau khi thêm : HienDS(list);
- Xóa nút đầu : XoaNutDau(list, a);
- Hiện danh sách sau khi xóa nút đầu : HienDS(list);
- Xóa Danh sách : XoaDS(list);

Chú ý : khai báo thêm biến int i,a, dung hàm system("pause"); dừng màn hình xem kết quả sau mỗi chức năng .